

Hackathon Problem Statements

→ The Dropout Signal

Problem context

Universities lose students to dropout every year — but the warning signs appear months earlier. Declining grades, missed assignments, and financial stress all precede the decision to leave. The challenge is not just prediction: it is fair prediction.

A model that disproportionately flags students from lower-income backgrounds or specific demographics is a liability, not a tool. Your pipeline must predict risk early, explain why, and prove it is not biased against any group.

Dataset — pre-loaded in your cluster

UCI Dropout Dataset [start here]

- **Path:** <https://www.kaggle.com/datasets/adilshamim8/predict-students-dropout-and-academic-success>
- **Size:** 4,400 records, 37 features — trains in seconds
- **Target:** Use binary: Dropout = 1, Enrolled/Graduate = 0
- **Contains:** Demographics, semester grades, financial indicators, macroeconomic context
- **Auth:** None — direct access, no login required

OULAD [only if you have spare time after hour 14]

- **Path:** https://analyse.kmi.open.ac.uk/open_dataset
- **Size:** ~32,000 students, 7 interlinked tables
- **Warning:** Multi-table join takes 3–4 hours alone. Skip unless your core pipeline is done and clock says < hour 14.

Enrichment sources — optional, after hour 14 only

- AISHE India Education Data: <https://aishe.gov.in> — district-level dropout rates, India context
- World Bank Education Stats: <https://datatopics.worldbank.org/education/> — macroeconomic context
- Kaggle education datasets: <https://www.kaggle.com/datasets?search=student+dropout>

What you must build — in priority order

Build in this order. Stop when time runs out. A clean 4-step pipeline with a documented fairness audit beats a 7-step pipeline that never reaches fairness.

1. **Bronze layer:** Ingest uci_dropout.csv into a Bronze Delta table. Document schema.
2. **Silver layer:** Clean nulls, engineer features: semester-on-semester grade delta, absenteeism trend, financial stress index.
3. **Two models:** Logistic Regression baseline + one tree model (Random Forest or XGBoost). Log both runs in MLflow. Register the better one in Model Registry.
4. **Fairness audit:** Compute demographic parity and equal opportunity across gender and socioeconomic group. Log metrics as a Delta table. Document any disparity found — do not hide it.
5. **SHAP explainability:** Run SHAP on your best model. Surface top 3 risk factors per flagged student.
6. **Gold output table:** At-risk students: student ID, risk score, top 3 factors, recommended intervention tier (low / medium / high).
7. **[Bonus] OULAD enrichment:** If time allows after hour 14: join VLE engagement logs to add clickstream-based engagement features.

Databricks stack

- Delta Lake — Bronze / Silver / Gold medallion architecture
- MLflow — experiment tracking, model registry
- Unity Catalog — Gold table registration

- SHAP — pip install shap (explainability)
- Databricks AutoML — optional bonus benchmark

Evaluation criteria

Criteria	Weight
Pipeline architecture — Bronze / Silver / Gold discipline	20%
Feature engineering depth & creativity	20%
Model performance & MLflow experiment hygiene	20%
Fairness audit — metrics, documentation, honest findings	25%
Explainability & Gold table output quality	15%

→ The Drifting Oracle

Problem Context

A credit scoring model trained on historical data is now scoring applicants in a post-inflation economy — feature distributions have shifted and nobody noticed. Simultaneously, an LLM explaining loan rejections in plain English has started generating plausible-sounding but legally non-existent justifications. Both problems need monitoring systems, not patches.

Datasets

Name	Link	Details
Home Credit Default Risk	Kaggle	350K loan applications, rich feature set — use as training distribution
German Credit Dataset	UCI ML Repository	1K records, different feature distribution — use as simulated drift batch
RBI / SEBI Guidelines Reference	RBI Notifications	Provide teams a curated list of real regulations as ground truth for LLM eval

Objectives

1. **Baseline Model** — Train a credit risk classifier on Home Credit data; log all params, metrics, and artifacts with MLflow; register as **Champion** in Model Registry

2. **Drift Detection** — Score the German Credit batch; compute PSI (Population Stability Index) on at least 5 key features; write drift metrics to a Delta table
3. **Retraining Trigger** — When PSI exceeds threshold on any feature, trigger automated retraining; register new model as `Challenger` ; compare Champion vs Challenger using `mlflow.evaluate()`
4. **LLM Evaluation Pipeline** — Build a pipeline using `mlflow.evaluate()` with custom metrics that score each loan rejection explanation for:
 - Factual grounding (does it reference actual data?)
 - Hallucination (does it cite a regulation not in the reference list?)
5. **Governance** — All monitoring outputs (drift metrics, eval scores, retraining logs) must land in governed Unity Catalog Delta tables

Databricks Stack

- MLflow (experiment tracking, model registry, `mlflow.evaluate()`)
- Delta Lake (feature store, monitoring tables)
- Databricks Model Serving (LLM endpoint)
- Unity Catalog (governance of monitoring outputs)

Evaluation Criteria

Criteria	Weight
Drift detection methodology & PSI implementation	25%
Retraining trigger logic & Champion/Challenger comparison	25%
LLM eval pipeline with custom hallucination metrics	25%
MLflow experiment hygiene & registry discipline	15%
Unity Catalog governance of all outputs	10%

 **Evaluator Note:** Push teams on their PSI threshold choice — why that number? Also ask how their LLM eval would scale to 10,000 explanations per day.

→ Fire Risk Prevention & Response Optimization Engine

Complexity Level: Advanced

Real-World Application: City of San Francisco Fire Department Operations

Project Objective

Build an integrated data platform that synthesizes 14 days of analysis into a production-ready system supporting fire department strategic planning, proactive compliance, and resource optimization.

Datasets Overview

All datasets are pre-loaded into Databricks from San Francisco Open Data portal ([Socrata API](#)).

▼ 1. Fire Incidents Dataset

Source: SF Fire Department

Records: 250,000+ incidents

Key Fields:

- `incident_number` - Unique incident identifier
- `call_number` - Associated CAD call number
- `incident_date` - Date/time of incident
- `incident_type` - Classification (Structure Fire, Alarm, etc.)
- `address` - Property address
- `district` - Fire department district
- `neighborhood` - Neighborhood district
- `property_loss` - Estimated financial loss
- `units_responding` - Number of fire units dispatched
- `response_time_minutes` - Time from call to arrival

Purpose: Core incident data for analysis of fire department response patterns and incident distribution across the city.

▼ 2. Fire Calls-for-Service Dataset

Source: SF Fire Department CAD System

Records: 1,000,000+ call records

Key Fields:

- `call_number` - Unique call identifier
- `incident_number` - Associated incident
- `call_type` - Category (Medical, Alarm, Structure Fire, Traffic Collision, etc.)
- `received_dttm` - Time call received
- `dispatch_dttm` - Time units dispatched
- `response_dttm` - Time units arrived
- `unit_id` - Responding fire unit identifier
- `station_area` - Fire station area responsible
- `disposition` - Call outcome
- `address` - Call location

Purpose: Detailed call-level data showing response timeline, unit assignments, and call processing efficiency. Multiple records per incident (one per responding unit).

▼ 3. Fire Violations Dataset

Source: SF Fire Department Compliance Division

Records: 150,000+ violation records

Key Fields:

- `violation_number` - Unique violation identifier
- `address` - Property address
- `violation_date` - Date violation issued
- `violation_type` - Category (Fire Code violation, hazard, etc.)
- `fire_code_reference` - Specific code section violated
- `violation_status` - Status (Open, Closed, In Progress)

- `severity_level` - Severity classification
- `inspection_number` - Associated inspection ID
- `district` - District location

Purpose: Compliance and enforcement data showing safety violations identified during inspections. Critical for identifying high-risk properties and repeat offenders.

▼ 4. Fire Inspections Dataset

Source: SF Fire Department Inspections Program

Records: 80,000+ inspection records

Key Fields:

- `inspection_number` - Unique inspection identifier
- `address` - Inspected property
- `inspection_type` - Type (Initial, Follow-up, Complaint Response, etc.)
- `inspection_date` - Date inspection performed
- `start_dttm` - Inspection start time
- `end_dttm` - Inspection end time
- `inspector_id` - Inspector assigned
- `disposition` - Result (Passed, Failed, Violations Found, etc.)
- `district` - District location
- `violations_found` - Count of violations identified

Purpose: Preventive inspection data showing building compliance checks and inspection outcomes. Links to violations dataset.

▼ 5. Fire Permits Dataset

Source: SF Fire Department Permits Division

Records: 50,000+ permit records

Key Fields:

- `permit_number` - Unique permit identifier
- `address` - Property address

- `permit_type` - Type (Fire Protection System, Alarm, etc.)
- `permit_start_date` - Permit issue date
- `permit_expiration_date` - Expiration date
- `permit_status` - Status (Active, Expired, Suspended, etc.)
- `special_conditions` - Required conditions or restrictions
- `district` - District location

Purpose: Permits and authorizations data showing fire safety system installations and special conditions. Indicates properties with active fire protection measures.

Capstone Deliverables

1. **Risk Prediction Layer** - Predictive models identifying high-risk properties and neighborhoods
2. **Response Optimization Layer** - Data-driven resource allocation recommendations
3. **Compliance Tracking System** - Violation follow-up effectiveness measurement
4. **Operational Dashboard Dataset** - Aggregated metrics for decision-makers

Expected Outcomes

- 5+ optimized Delta tables in Gold layer
- Comprehensive technical documentation
- Executive summary with actionable recommendations
- Code repository and reproducible pipeline